

Backend Engineer Journey – Day 4 (OOP Foundations)

1. Why Object-Oriented Programming (OOP)?

Before OOP, procedural programming stored data and functions separately, making large applications difficult to maintain.

OOP solves this by combining **data (state)** and **behavior (methods)** into a single unit called an **Object**.

Benefits of OOP

- Better code organization
 - Reusability
 - Easy maintenance
 - Scalability for large applications
 - Real-world modeling
 - Easier teamwork in enterprise applications
-

2. Class

A **Class** is a blueprint or template used to create objects.

It defines:

- Properties (Fields/Variables)
- Behaviors (Methods)

Example:

```
class Customer {  
  
    String name;  
    String email;  
  
    void buyPolicy() {  
  
    }  
  
}
```

A class itself is **not a real object**.

3. Object

An **Object** is a real instance of a class.

Example:

```
Customer c1 = new Customer();  
  
Customer c2 = new Customer();
```

Here:

- One Class
- Two Objects

Each object has its own memory.

4. Class vs Object

Class	Object
Blueprint	Real Instance
Template	Actual Entity
Defines structure	Stores actual data
Used to create objects	Created using <code>new</code>

5. Object Memory

Example:

```
Student s = new Student();
```

Memory:

Stack

s

↓

Heap

Student Object

- Reference variable (s) → Stack
- Object → Heap

6. Three Characteristics of an Object

State

Current data of an object.

Example:

```
Name = Sameer  
Age = 22  
Email = abc@gmail.com
```

State is represented by instance variables.

Behavior

What an object can do.

Examples:

```
study()  
  
login()  
  
buyPolicy()  
  
renewPolicy()
```

```
withdraw()
```

Behavior is represented by methods.

Identity

Every object has its own unique identity.

Even if two objects have exactly the same data, they are still different objects.

Example:

```
Student s1 = new Student();  
  
Student s2 = new Student();
```

Both are different objects even if their field values are identical.

7. OOP Principle

OOP combines:

- Data (State)
- Behavior (Methods)

inside a single object.

Instead of writing separate functions for everything, objects manage their own data and actions.

8. Spring Boot Connection

Almost everything in Spring Boot is based on OOP.

Examples:

```
UserController  
  
UserService
```

UserRepository

These are classes.

During application startup, Spring creates objects (called **Beans**) from these classes and manages them.

9. Important Memory Concepts

- Objects are created in Heap memory.
- Reference variables are stored in Stack memory.
- One class can create multiple objects.
- Every object has its own separate memory.
- Multiple references can point to the same object.

Interview Takeaways

- OOP models real-world entities.
- A class is a blueprint, not an object.
- An object is an instance of a class.
- Every object has:
 - State
 - Behavior
 - Identity
- One class can create many objects.
- Objects are stored in Heap memory.
- Reference variables are stored in Stack memory.
- Spring Beans are objects managed by the Spring Framework.

Golden Lines (Remember Forever)

- A class defines what an object **can have** and **can do**.
 - An object is a real instance of a class.
 - State = Data.
 - Behavior = Methods.
 - Identity = Every object is unique.
 - One class can create unlimited objects.
 - OOP combines data and behavior into a single unit.
 - Spring Boot is built on OOP principles.
-

Common Interview Traps

✗ "A class is an object."

✓ Incorrect.

A class is a blueprint. Objects are created from it.

✗ "Two objects with the same data are the same object."

✓ Incorrect.

They have the same state but different identities.

✗ "Spring Bean is a class."

✓ Incorrect.

A Spring Bean is an **object** created and managed by the Spring Framework.

Real Project Connection (PolicyHub)

Customer

State

- customerId
- name
- email
- phone
- address
- status

Behavior

- register()
- login()
- buyPolicy()
- renewPolicy()
- updateProfile()
- viewPolicies()

Identity

- `customerId`

This is exactly how we model real enterprise applications.

Day 4 Summary

By the end of Day 4, you should be able to explain:

- Why OOP exists.
- Difference between Class and Object.
- State, Behavior, and Identity.
- How objects are created.
- Why Spring Boot is built on OOP.
- How to think about designing real-world software using classes and objects.